

Lean Tutorial at TIFR

Sidharth Hariharan

sidharthhariharan@cmu.edu

3 and 4 August, 2026

1 Introduction

Lean is an interactive theorem prover that has been around for over a decade. The promise of an interactive theorem prover is to formally verify the correctness of a mathematical proof, which has led to a dramatic rise in their popularity since the advent of AI. Interactive theorem provers far predate AI, though, and have long benefitted those using them in a wide variety of ways (see [\[1\]](#), for instance).

Lean gained significant attention from the mathematical community when Kevin Buzzard, Johan Commelin and Patrick Massot led an effort to [formalise the definition of a perfectoid space](#) in 2019. Shortly afterwards, a collaboration led by Johan Commelin in response to a challenge by Peter Scholze led to the completion of the [Liquid Tensor Experiment](#), which gathered attention even more attention from within and beyond mathematics.

A lot of mathematics has been formalised since then (and indeed, a lot was formalised prior to it as well). Much of the mathematics that has been formalised in Lean is organised into a massive, community-maintained library called [Mathlib](#). Many projects also exist downstream of Mathlib, such as [Fermat's Last Theorem](#), the [Carleson project](#), the [Equational Theories project](#), the [Infinity Cosmos project](#), and the [Sphere Packing project](#) (which I have been leading since February 2024). A more complete list can be found [here](#).

This four-part tutorial series seeks to provide a brief but practical introduction to theorem proving in Lean. I will aim to cover the basics of thinking in Lean, proving basic theorems,

searching through Mathlib, different design choices in the library, and effectively using off-the-shelf AI tools to prove theorems in Lean. Time-permitting, I will also share some information about how to set up/contribute to community projects.

2 Lean on the Web

An easy, lightweight solution to using Lean is to use the web version. I do not recommend this for long-term Lean use, but it is very convenient for writing Lean on the fly. Visit <https://live.lean-lang.org> to get started.

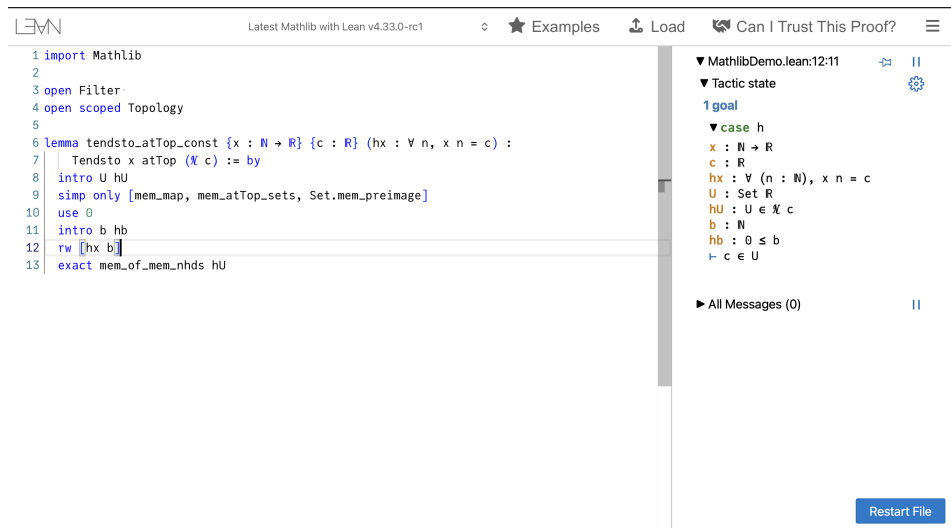


Figure 1: The Lean web interface

Pro tip: if you’ve typed out some code that you’d like to share, you can simply copy and paste the URL. For instance, the code in the above screenshot can be found [here](https://lean-lang.org/install/).

3 Installing Lean

It is strongly recommended that you install (or attempt to install) Lean in advance of the first session. Instructions to do so can be found here:

<https://lean-lang.org/install/>

I strongly recommend installing Lean through the Visual Studio Code extension, which I’ll describe below in brief. For other ways of installing Lean, visit the link above.

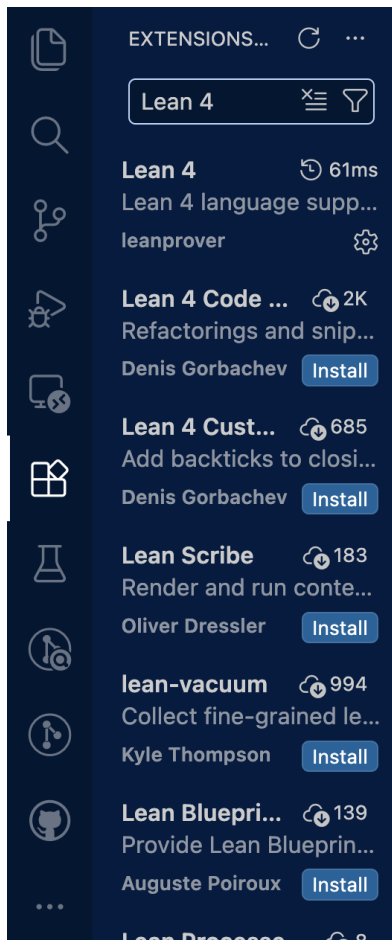


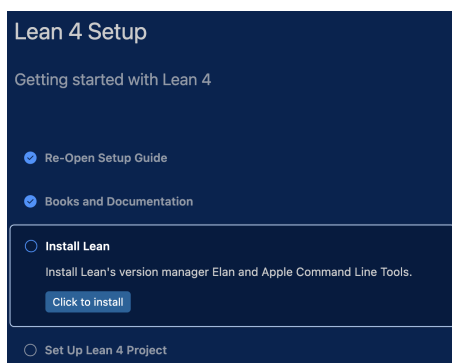
Figure 2: The extension

in Figure 3a, it says “Apple Command Line Tools” because I have a Mac). You can also go set up a Lean 4 project if you like, but it won’t be necessary for these tutorials.

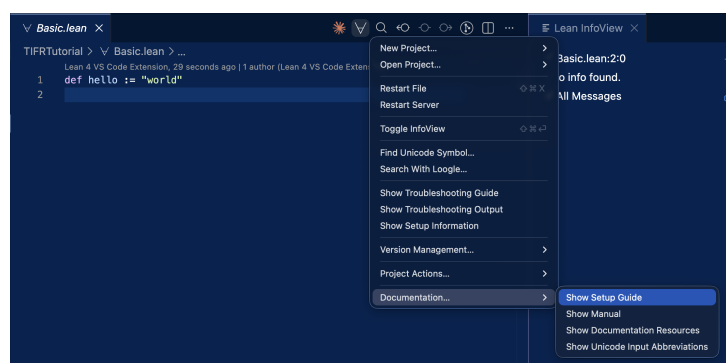
Install VS Code (if you don’t have it) and launch it. Navigate to the panel on the left and click on “Extensions” (the 2×2 square with the top-right square rotated 45°). Type in “Lean 4”. Install the “Lean 4” extension authored by `leanprover` (top one in Figure 2). The “Install” button doesn’t appear in Figure 2 because I already have the extension.

Once you’ve installed the extension, a setup guide should open automatically. See Figure 3a. If this does not happen, you can still open it manually by opening an empty file, clicking on the ∇ -symbol in the top right, and then selecting ‘Documentation...’ $\rightarrow$ ‘Show Setup Guide’. See Section 3.

Click on the “Click to install” button under the “Install Lean” section of the guide. This will require internet connection. While this should not objectively be problematic, this is also the step where you are most likely to encounter problems. What should happen is that it should install `elan`, the Lean version manager, and possibly additional tools (for instance,



(a) The extension setup guide



(b) Reopening the extension setup guide

Figure 3

4 The Plan

First of all, I have created a GitHub repository for this tutorial:

<https://github.com/thefundamentaltheor3m/TIFR-Tutorial>

Instructions on setting it up can be found in the repository README. The TLDR is that you want to clone the repository, go into the project folder, and run

```
lake exe cache get
```

before doing anything. This will download the Mathlib cache and populate the project's `.lake` folder with it. (This folder isn't visible on GitHub as it's `.gitignored`, but it will be visible locally once you get cache.)

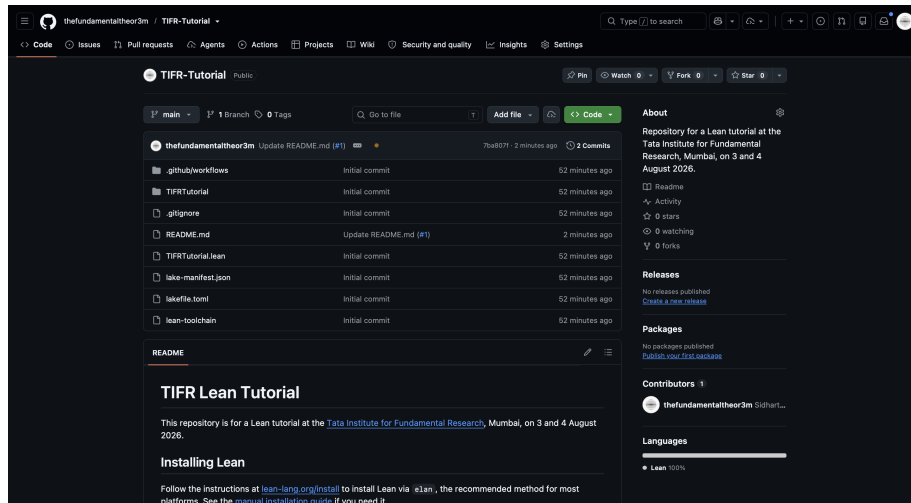


Figure 4: Tutorial repository

My rough plan is to have four sessions. They will be centred on the following themes.

1. **Getting Oriented:** Propositions as types, proofs as terms, simple tactics and how to use them.
2. **Structure and Design:** What do libraries look like? How are different notions of “structure” captured? How are definitions written? Taking a walk through Mathlib and exploring and interfacing with what’s in there.
3. **How to Lean:** Common conventions, how to search for theorems, how to effectively

use AI tools.

4. **Leaning Together:** Interactively working through the proofs of simple theorems.

We will convene at the following times:

1. **Monday, 3 August:** 10:00 to 12:00 (break from 1:50 to 11:10)
2. **Monday, 3 August:** 14:00 to 16:00 (break from 14:50 to 15:10)
3. **Tuesday, 4 August:** 09:30 to 11:30 (break from 10:20 to 10:40)
4. **Tuesday, 4 August:** 13:30 to 15:30 (break from 14:20 to 14:40)

On each day, will have a morning session (10:00 to 12:00 with a break from 10:50 to 11:10) and an afternoon session (14:00 to 16:00 with a break from 14:50 to 15:10). I'll set aside some time during each session for discussion and exercises.

I want to keep this very informal. We can always alter course as per popular demand. If we have time to do more, we can do more (I have plenty to say), but I don't want to do something so rushed that it isn't helpful. I'm very open to suggestions, comments, requests... feel free to write me an email at sidharthhariharan@cmu.edu.

One disclaimer: I'm not a computer science expert, and I'm still a graduate student in mathematics. While I do have experience in formalisation, there's a more than decent chance I may not know the answers to some of your questions. That being said, I'll do my best.

5 Leaning In

One obvious way to Lean into the hype is to come to the tutorial. But there are many more ways you can get started.

The Lean Game Server is an excellent resource:

<https://adam.math.hhu.de>

There are loads of games you can play here. The one I most strongly recommend is the

Natural Number Game, some ancient Lean 3 ancestor of which I played when I first learnt Lean. Others include Alex Kontorovich’s “Real Analysis, The Game” and Emily Riehl’s “A Reintroduction to Proofs” (for my type-theoretically inclined friends).

See you soon! And in the meantime...

HAPPY LEANING!

References

- [1] Jeremy Avigad. *Mathematics and the formal turn*. 2023. arXiv: [2311.00007](https://arxiv.org/abs/2311.00007) [math.HO].
URL: <https://arxiv.org/abs/2311.00007>.